

SVKM's Narsee Monjee College of Commerce and Economics
(Autonomous)

TeachMate: A Teacher's Helper

A Project Submitted to
Department of Information Technology

By

A034 45618230001 Stasha Pereira

Under the Guidance of

Dr. Anupama Jawale

Abstract

Education is still one of the most thought-provoking and socially influential occupations of the day. It demands not only the expertise in the area of the subject matter, but also the skills to handle dynamic classrooms, to arouse interest and offer individualized instructions to learners with learning needs. The teaching profession, however, is being defined by increasing numbers of students, increasing curricular demands and the increasing and mounting demands of individualized education. These elements give an administrative load which tends to overwhelm the creative and human side of teaching. Activities like lesson plans, marking of works, creation of work sheets, writing of progress reports, and constant communication with parents require a lot of time and energy.

The main aim of this paper is to provide an introduction to TeachMate - A Teachers Helper, an AI-powered teaching assistant platform that is specifically organized to overcome these issues. In contrast to the traditional EdTech solutions, which mostly focus on the student learning outcomes, TeachMate aims at assisting teachers by automating daily procedures, yet still leaving room to exercise professional judgment. Its main characteristics are developing personalized lesson plans, performance dashboards, the modules of weakness analysis, and automatic feedback reporting systems. With them, the system does not only simplify the process of administration, but also makes the students get academic assistance in the most timely and intentional manner.

Notably, TeachMate is not trying to substitute teachers, which is usual in the discourse of AI in education, but rather a facilitator of human-centered pedagogy. With the removal of repetitive activities by technology, teachers are relieved to attend to activities that are of higher value, including mentoring, solving problems, and nurturing learner creativity. The paper will talk about the motivation behind the system, the methodology behind the system, experimental validation of the system through pilot testing, hardware and software requirements, system outputs, and what could be improved in the future. By doing so, it adds to the current debates about the responsible integration of the use of artificial intelligence into classrooms and its ability to reduce the central role of the teacher.[1][2]

Introduction

The 21st century has experienced rapid changes in education because of globalization, digitalization and changes in demands of the workforce. The current day classroom is much more heterogeneous and students have different backgrounds, learning rates, and prior knowledge levels. Not only are the teachers expected to impart the content of the curriculum but also to inculcate critical thinking, creativity, emotional intelligence, and problem solving skills in their students. These obligations are exaggerated by the necessity to correspond to the standards in the institution, track the progress of the students and keep the communication with parents effective.

The workload of a regular teacher goes much beyond the teaching time in the classroom. Literature indicates that teachers dedicate much of their time beyond the school hours in planning lessons, marking papers and documentations. An example is the time spent to prepare a week of lessons to more than one grade level which may require several hours. On the same note, it is time-consuming and mentally draining to create individualized progress reports of dozens of students. This continual balancing of administrative and pedagogical tasks, can often cause burnout in teachers and decrease classroom productivity.

The following are some of the essential issues that teachers have to endure:

1. **Time Management:** Teachers should find a balance between preparation, delivery and evaluation. The work of the administration usually runs over into the personal area, leaving time to create creative lessons.
2. **Different Student Needs:** Within one classroom, there might be highly gifted learners, middle achievers, and those students who need remedial intervention. It is challenging to serve this range using scarce resources.
3. **Performance Tracking:** Teachers have to keep accurate records of student achievement in various courses and assessments. Manual or spread sheets are ineffective and prone to errors.
4. **Communication Ineffectiveness:** Parents want to be informed about the academic progress of the child promptly. But teachers are frequently too busy to give comprehensive and in-depth communication, which is individualized.

It is here where TeachMate - A Teachers Helper comes in very handy. It is built as a personal digital assistant of teachers, and it provides intelligent automation when it comes to repetitive tasks and analytics when it comes to informed decision making. TeachMate is an empowerment-oriented platform, unlike many digital platforms that only concentrate on the experience of the learner. Its features liberate teachers of the burden of administration and assist them in devoting more time to the human side of teaching-mentorship, motivation and interactive learning.

TeachMate can show that artificial intelligence can be introduced to the world of education ethically and realistically by being a complementary partner and not a substitute. It emphasizes how technology in tandem with pedagogy can guarantee efficiency as well as equity and individualization of learning.[4]

Related Work

EdTech in particular has experienced a tenfold increase in less than 10 years, in large part thanks to the innovation of cloud computing, machine learning, and access via mobile devices. There are numerous websites that serve the purposes of teachers and learners. The unifying weakness in most of these solutions is, however, also their student-centric nature- they expect to facilitate student learning directly but fail to offer the teachers sufficiently tools to alleviate the burden of teachers, who are central to the educational process.[5]

A further discussion of commonly used EdTech applications and their shortcomings as related to teacher support follows below:

1. Google Classroom

- Strengths: Smooths out assigning, grading and classroom announcements. Comes with additional Google applications like drive and Docs, which enable ease of sharing materials.
- Limitations: It is rather a classroom management system than an assistant of a teacher. Educators continue to have to design the lesson manually, analyze data on student performance, and prepare the progress reports. Insights based on AI are limited.

2. Khan Academy

- Strengths: Offers free, customized learning materials, adaptive quizzes and practice exercises to students. The teachers can track the progress of students using its data dashboards.
- Limitations: The teachers are allowed to see progress, although they still need to manually interpret performance data to see the weaknesses. The site lacks automatic lesson customization and communication with parents.

3. Coursera and EdX (MOOCs)

- Strengths: Provide massive access to higher and professional education. Deliver online classes, exams, and qualifications to millions of students around the globe.
- Limitations: These sites are intended as independent learning platforms, but not classroom management. School teachers receive little direct support on such systems on issues of workload reduction.

4. Intelligent Tools (ITS).

- Strengths Use AI algorithms to adjust lessons to student learning rate and deliver instant feedback. There are systems that are able to do real time analysis of student misconceptions.
- Limitations: They are too biased on the adaptation of students as opposed to teacher empowerment. Although ITS is capable of pointing out weaknesses in the students, it does not provide teacher-friendly reports or administrative tasks such as lesson planning or communicating with parents.

5. Learning Management Systems (LMS) like Moodle, Blackboard and Canvas.

- Strengths: Offer online content delivery, grading and collaboration. Provide schedule offer programs and web-based evaluation programs.

- Downsides: These are infrastructure-based platforms, these provide the digital framework, but the actual input and analysis/management of the data is left to the teacher. They do not have high level AI-driven automation to minimize repetitive tasks.

Comparison to TeachMate

TeachMate stands out as one which specifically focuses on the teacher side. Its design principles extend further than the support of student learning, it includes:

- Accurate lesson planning which is automated and less time consuming.
- Data analytics and visualization tools were aimed at assisting teachers with learning about the class-level and individual-level performance.
- Automation of parent communication, which saves time that the teachers would otherwise use to write progress reports.

Therefore, TeachMate fills a void left by current systems: it is not only a tool that helps students but a real helper to teachers, which makes it more comprehensive in the context of the modern classroom.[3]

Methods

TeachMate is developed on the basis of artificial intelligence, data analytics and communication technologies. It is a modular system that helps to be scalable and flexible in the various school settings. The 4 key modules are explained below in detail.

1. Data Collection Module

- Data Sources: Gathers student records including demographic information, attendance, grades, assignments and even behavioral notes provided by teachers.
- Multi-Input System: Data may be entered manually by the teachers, or may be integrated with preexisting school databases and Learning Management Systems.
- Data Formats: The structured data (marks, attendance percentages) and unstructured data (teacher comments, qualitative observations) are supported.
- Preprocessing: Data cleaning, normalization, as well as tokenization are used in preprocessing to prepare the inputs to be analyzed.
- Output: Gives a student profile in detail, upon which lesson customization and prediction of performance is based.

2. Lesson Planning Module

- Curriculum Alignment: The lesson plans are aligned to state or national curriculum standards in order to make sure that the standards are complied with.
- Adaptive Planning: The system proposes differentiated activities- remedial activities to weaker students, enrichment activities to the better students.
- Editable Templates: Teachers have access to editable lesson outlines to have full control, and to make sure that there is a human perspective of AI suggestions.
- Resource Integration: Helps to include multimedia resources (videos, slides, interactive activities), which the AI proposes to increase the lessons.
- Result: It saves teachers a lot of time on curriculum preparation and yet still offers it relevance and flexibility.

3. Weakness Analysis and Performance Tracking Module.

- Machine Learning Models: This applies methods of classifications and regression to make predictions about the student performance patterns.
- Visualization Tools: This creates dashboards in graph, charts, and heat maps that are easily interpreted on the performance data.
- At-Risk Identification: Warns students who are not performing on the average of the rest of the class or whose performance is in downward trends.
- Class Insights: Compiles data into trends and identifies the areas that an entire class is poorly performing (e.g., fractions in mathematics, grammar in English).
- Outcome: Allows teachers to take action at an early stage of specific strategies.

4. Feedback Module and Communication.

- Automated Report Creation: Generates personalized progress reports with a summary of strengths, weaknesses, attendance and performance.
- Parent Notifications: Sends Notifications to parents through email or SMS, making sure that they are heard in time.
- Teacher Customization: Provides the option of teachers to include personal notes, remarks or encouraging comments, in addition to the AI-generated insights.
- Consistency: Provides parents with consistent, data-based feedback that is not different in quality or detail.
- Result: This decreases teacher work and enhances trust and transparency between parents.

Experiments

In order to confirm the feasibility and efficacy of TeachMate - A Teacher Helper, a pilot study of small scale was carried out in a school environment. The experiment targeted a single teacher and some students rather than the systematic application of the system to a large number of classrooms. This option made it possible to have deep monitoring, close observation and repetitive feedback, which is needed at the initial phases of system evaluation.

Participants

1. **Teacher:** A middle school teacher who teaches several subjects (Mathematics and Science) of various grades.
2. **Students:** Grade 6 to 10: a small group of students who will be chosen to represent the increasing levels of academic levels:
 - Students who have been performing well academically, and are generally good in classwork and assignments.
 - Average performers, who perform to minimum grade expectation but are inconsistent.
 - Students that need special attention, who often have trouble grasping some concepts.

Such a participant structure also guaranteed the adaptability of TeachMate to various learner profiles to be tested successfully.

Duration and Setup

1. **Period:** The experiment was conducted in a period of a week and a half.
2. **Topics Addressed:** Mathematics (investigations of fractions, algebra, geometry) and Science (investigations of the basics of biology and physics).

Implementation:

TeachMate was used by the teacher to:

- Coming up with weekly lesson plans.
- Monitoring student performance of quizzes and assignments.
- Going through the weaknesses via dashboards.
- Giving feedback messages to students and parents.

Procedure

1. **Baseline Observation:** During the first week, the teacher documented the time taken to plan the lesson, grade and communicate using the traditional methods. This served as a benchmark.
2. **TeachMate Integration:** Starting with the second week, the teacher switched to TeachMate to plan his lesson and track his students.
3. **Student Engagement:** The students were presented with lesson plans that were created with the help of TeachMate and with assessments tracked by the system.
4. **Feedback Collection:** The teacher and students gave feedback about the usability and effectiveness of the system at the end of every week.

Findings

1. Teacher's Perspective

- Lesson Planning: The teacher noted that there was a major decrease in the time spent on planning, which was approximately 6-7 hours per week to approximately 3-4 hours since TeachMate offered editable templates that were used in accordance with the curriculum objectives.
- Performance Insights: Through the dashboards, the teacher was able to determine immediately that the grade 7 students were having a harder time with fractions, whereas students in grade 9 were having a harder time with physics-based problem solving.
- Administrative Relief: Automated reports on progress saved the teacher the burden of having to write a progress report on each student one at a time which used to take the teacher several evenings.

2. Students' Perspective

- Transparency of Instruction: Students valued systematic lesson plans and clear learning objectives and practice tasks based on their level of performance.
- Motivation: Students with high performance were motivated with enrichment activities and underperforming students felt motivated with being offered specific remedial works rather than generic worksheets.
- Reception of Feedback: Students stated that prompt feedback (given on a weekly basis) provided them with more sense of direction in their studies.

3. Parents' Perspective

- Parents appreciated the regularity of the updates done via email/SMS. They were also more engaged in the education of their child without depending on inconsistent parent-teachers meetings.
- Reporting was easier and simpler to analyze through the graphical portrayal of student advancement.

Hardware & Software Requirements

1. Hardware Requirements:

- Processor: Intel i5 / AMD Ryzen 5 (minimum)
- RAM: 8 GB (16 GB recommended for AI processing)
- Storage: 256 GB SSD (expandable)
- Internet: Reliable broadband connection
- Optional: Cloud infrastructure for large-scale deployment

2. Software Requirements:

- **Operating System:** Windows 10/Linux/macOS
- **Programming Languages:** Python (for AI, data processing), JavaScript (frontend),
- **API:** Ollama
- **Frameworks:** TensorFlow / PyTorch (ML), Flask/Django (backend),
- **Databases:** SQLAlchemy,
- **Visualization Tools:** Matplotlib, D3.js, or Plotly for dashboards
- **Communication APIs:** SMTP (email), SMS gateways for parent communication

Output

Those are not mere reports or dashboard generated by TeachMate - A Teacher Helper; they are actionable and data-driven insights that enhance teaching and learning. The key types of outputs and their characteristics are the following:

1. Customized Lesson Plans

- **Artificial Intelligence Generated Outlines:** The system will generate lesson plans based on the subject, level of the class, and the learning outcomes. An example of this can be seen in a grade 8 mathematics lesson plan that focuses on solving algebraic problems, whereas a grade 6 mathematics lesson plan will focus on the foundations of arithmetic.
- **Differentiated Learning Support:** Students with higher understanding are given enrichment instruction like higher sets of problems but the students with poor understanding get remedial exercises.
- **Flexibility in Teachers:** Teachers are allowed to examine and update these plans, and include their own notes, their favorite teaching methods, or classroom tasks. This acts to make sure that the AI recommendations do not eclipse teacher knowledge.
- **Time Efficiency:** Teachers spend an average of several hours per week planning time, which can be spent in the classroom.

2. Weakness Analysis Reports

- **Personalized Feedback:** every participant will be given a report on their weak aspects which is individualized. As an example, a student may be a genius in geometry and cannot do fractions; TeachMate shows this area of weakness to improve on it.
- **Class-Wide Gap Analysis:** The system sums up the weaknesses of the students of a particular class to indicate that several students are having similar problems. This assists educators to restructure lesson presentation.
- **Suggestions on how to become better:** The reports contain recommendations, e.g. additional practice sheets, additional videos, or certain subjects to repeat.
- **Comparative Benchmarking:** The weakness analysis is also a comparison of student performance to the average or historical performance of the class, whether the student is improving at a faster or slower rate than the other students or at the same rate.

3. Automated Feedback Reports

- **Parent Communication:** Progress report is automatically created and sent to parents to either email, this saves time on communication.
- **Detailed Formatting:** Reports contain both academic performance and attendance data as well as teacher feedback and AI-generated suggestions like study habits.
- **Consistency and Timeliness:** TeachMate provides timely and consistent updates to each parent in contrast to manual report writing that can be more or less detailed or frequent.

In sum, TeachMate will make traditional record-keeping an intelligent ecosystem of insights, which will help overcome the communication gap between teachers, students, and parents and will save a lot of time for the teachers.

Mentorly AI: Your Co-pilot in the Classroom

Focus on teaching — we'll handle the rest

Lesson Planning
Craft custom lessons & resources in minutes.

Smart Reports
Automate and track progress

Personalised student support
Upload an Excel to visualize class performance

Progress Report

Stasha

Biology

Photosynthesis

35

+ Add topic row

Improvement areas / next steps (optional)

Generate Report

Send now or schedule?

sapnapereira23@gmail.co

Send Now

dd-mm-yyyy --:--

Schedule

Progress Report for Stasha

Summary:

Stasha has made satisfactory progress in her photosynthesis module. With a score of 35, she demonstrates a solid understanding of

Send now or schedule?

sapnapereira23@gmail.co

Send Now

dd-mm-yyyy --:--

Schedule

Progress Report for Stasha

Summary:

Stasha has made satisfactory progress in her photosynthesis module. With a score of 35, she demonstrates a solid understanding of the process and its importance in plant growth. While there are no specific weaknesses or areas requiring improvement, it is essential to continue reinforcing her knowledge to ensure a deeper comprehension.

Subject-Specific Performance:

• Photosynthesis: 35

Recommendations & Measurable Goals:

To further improve Stasha's understanding of photosynthesis, I recommend the following:

1. ****Deep Dive into Light Reactions****: Encourage Stasha to explore the light-dependent reactions in more detail, focusing on the role of pigments and the importance of light intensity.
2. ****Explore Alternative Energy Sources****: Assign additional readings or activities that compare photosynthesis with other energy-producing processes, such as respiration or cellular respiration.

By achieving these goals, Stasha will solidify her understanding of photosynthesis and develop a more comprehensive perspective on plant growth and development.

Report Generated On:

March 15, 2023, at 14:30 hours

Mentorly AI: Your Co-pilot in the Classroom

Focus on teaching — we'll handle the rest

Lesson Planning
Craft custom lessons & resources in minutes.

Smart Reports
Automate and track progress

Personalised student support
Upload an Excel once — dashboard auto-updates from DB

Lesson Planner

Student Name (optional)

Math

4

Fractions

22-09-2025 21:54

40 mins

include visuals

Generate Lesson Plan

Here is the lesson plan:

****Lesson Plan****

****Date/Time:**** 2025-09-22T21:54

****Date/Time:**** 2025-09-22T21:54

****Subject:**** Math

****Topic:**** Fractions

****Grade:**** 4

****Duration:**** 40 mins

****Header:****

****Math Lesson****

Fractions

Grade 4

September 22, 2025 | 8:54 PM

Duration: 40 minutes

****Lesson Procedures (The "How-To"): Step-by-Step****

1. Introduction (5 minutes):

- * Review previous learning on basic addition and subtraction concepts.
- * Introduce the concept of fractions using real-life examples (e.g., pizza, cake).

2. Direct Instruction (15 minutes):

- * Define fractions as equal parts of a whole.
- * Use visual aids (diagrams, pictures) to illustrate:
 - + Identical parts: 1/2, 1/4
 - + Different parts: 3/4, 2/3
- * Explain the concept of equivalent fractions using examples (e.g., 1/2 = 2/4).

3. Guided Practice (10 minutes):

- * Provide students with a worksheet containing fraction problems.
- * Have them work in pairs to identify and write equivalent fractions.

4. Independent Practice (5 minutes):

4. Independent Practice (5 minutes):

- * Distribute a separate worksheet with more challenging fraction problems.
- * Ask students to solve on their own, using visual aids if needed.

****Assessment Ideas:****

* Formative Assessment:

- + Observe student participation during group work.
- + Review worksheets for understanding of equivalent fractions.

* Summative Assessment:

- + Administer a quiz or test at the end of the unit to assess students' understanding of fractions and equivalent ratios.

****Homework / Independent Practice:****

- * Assign students to complete a worksheet on fraction problems (e.g., adding, subtracting, multiplying).
- * Encourage them to create their own real-life examples of fractions and share with the class.

****Differentiation Tips:****

* For struggling students:

- + Provide additional visual aids or manipulatives (e.g., fraction strips) for support.
- + Offer one-on-one assistance during guided practice.

* For advanced students:

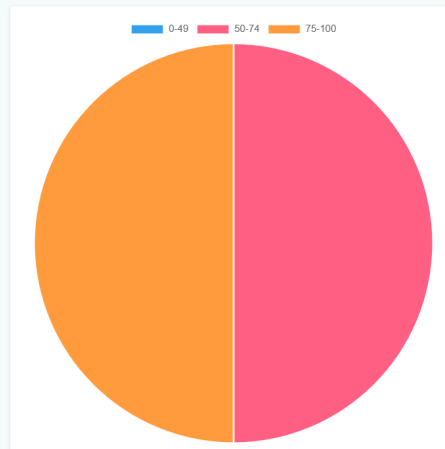
- + Challenge them to create their own equivalent fraction problems.
- + Encourage them to explore more complex fractions concepts (e.g., simplifying, comparing).

I hope this lesson plan meets your needs!

Choose file student_perf...template.xlsx

Upload & Ingest

✓ Ingested. Refreshing...



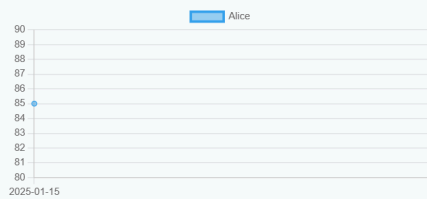
Alice

Alice

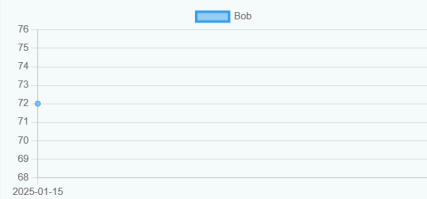
Bob

Bob

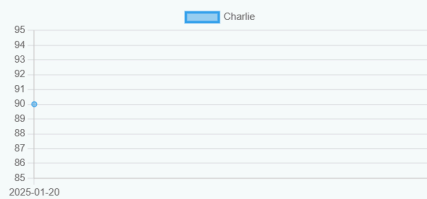
Alice



Bob



Charlie



Diana



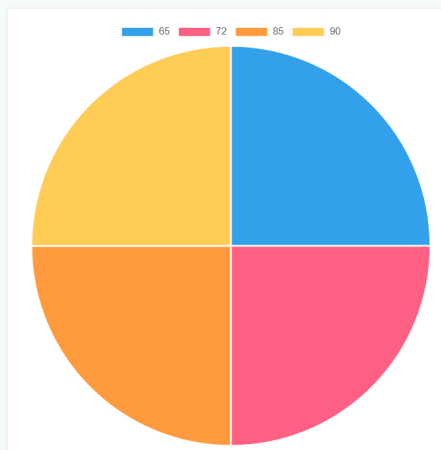
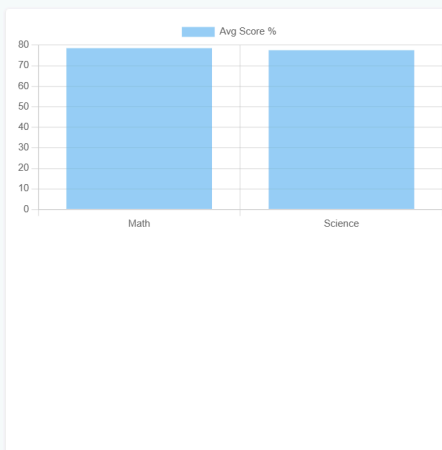
Top weak topics

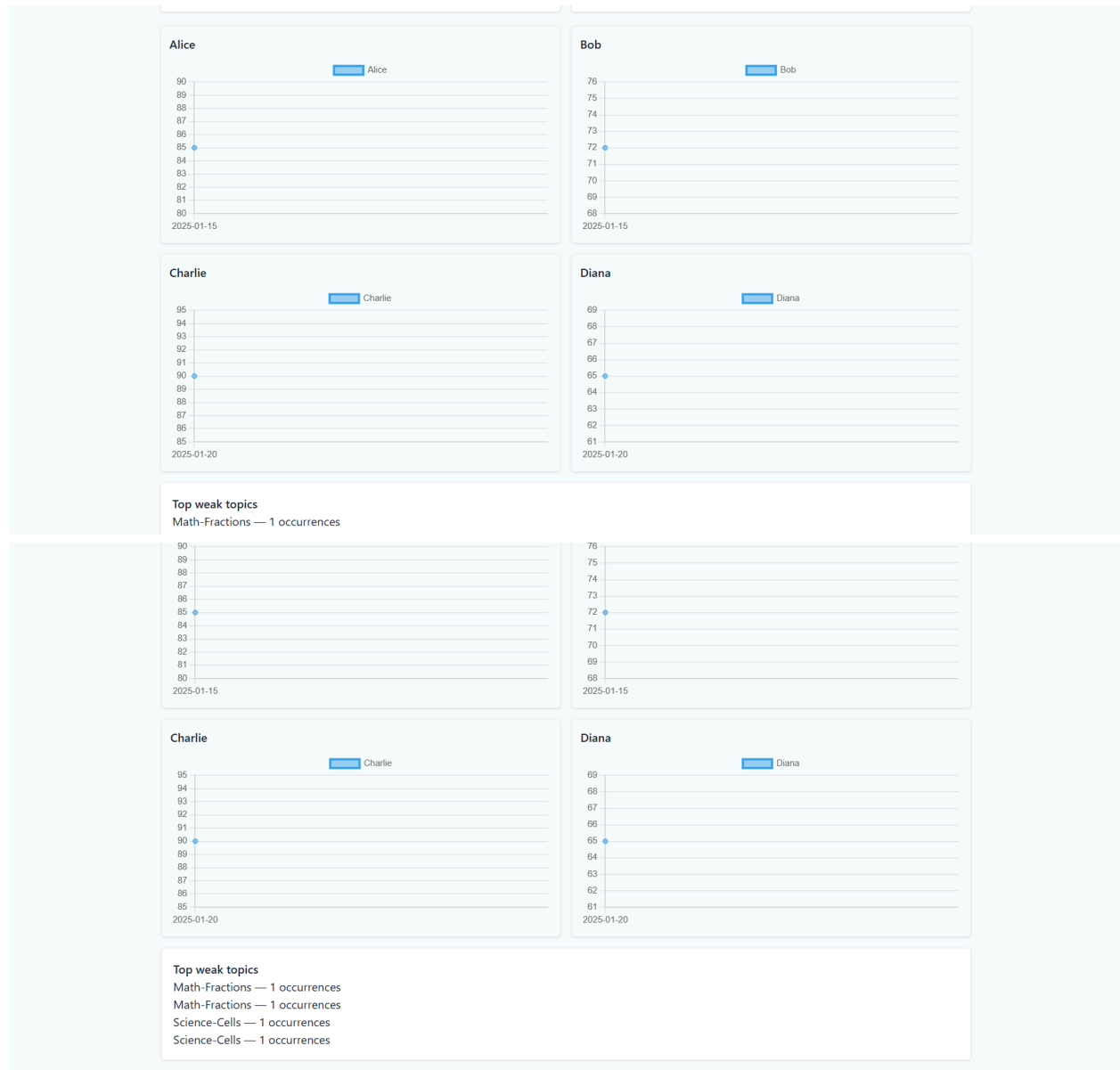
Performance Dashboard

Choose file student_perf...template.xlsx

Upload & Ingest

☒ Ingested. Refreshing...





Conclusion

The TeachMate - A Teacher Helper is developed to show that technology in well-considered designs can have a huge impact in the education sector. Instead of trying to substitute teachers, TeachMate is a partner company, which assists them in coping with the growing demands of contemporary classrooms.

TeachMate can enable teachers to:

- Free some time to interact with students and have classroom discussions.
- Invest in mentorship and conceptual clarity, the human-centered features of education, and AI will not be able to imitate them.
- Make evidence-based choices to make sure that interventions are specific and not generic.
- Enhance communication with parents, establish trust and make sure parents are informed of the progress made by the child.

The findings of the pilot study prove that TeachMate results in better efficiency, increased student engagement, and better collaboration between parents and teachers. Namely, educators were pleased by the opportunity to focus their energy on creative and interactive pedagogical activities instead of on administrative tasks.

In addition, TeachMate emphasizes a more general educational philosophy: AI in education must not rival human intelligence, it must complement it. Empathy, flexibility and inspiration are elements that a machine can never substitute because of their use by teachers. TeachMate enables teachers to do what they do best: empower their students with their abilities by completing regular tasks and concentrating on the areas that have the most significant impact.

To sum up, TeachMate is not a tool but a teaching pal, an important move towards more efficient, personalized and engaging classrooms in the future.

Future Scope

TeachMate is now concentrated on academic tracking and lesson planning, but there is significantly more to it. Improvements can be made in future, which may include:

- Correlation to National Curriculum Standards: Automatic correlation to regional or national education boards.
- Adaptive Learning Engines: More complex AI systems, personalizing content on the student-level.
- Predictive Analytics: Students at risk Preventive measures to detect students at risk of failure in school.
- Multilingual Support: Help of automatic translation of lesson plans into local languages to provide inclusive education.
- Mobile Application: allowing accessibility on the go by teachers and parents.
- Gamification Features: It involves making lesson plans more interactive and engaging to students.

With these features, TeachMate may become a fully fledged classroom ecosystem, filling gaps between teachers, students and parents.

References:

- [1] G. Brandhofer and K. Tengler, "Acceptance of AI applications among teachers and student teachers," *Discov Educ*, vol. 4, no. 1, p. 213, July 2025, doi: 10.1007/s44217-025-00637-w.
- [2] Ines Šarić-Grgić; Ani Grubišić; Slavomir Stankov; Maja Štula, "An agent-based intelligent tutoring systems review".
- [3] A. W. Fodouop Kouam, "The effectiveness of intelligent tutoring systems in supporting students with varying levels of programming experience," *Discov Educ*, vol. 3, no. 1, p. 278, Dec. 2024, doi: 10.1007/s44217-024-00385-3.
- [4] H. Yusuf, A. Money, and D. Daylamani-Zad, "Towards reducing teacher burden in Performance-Based assessments using aivalue: an emotionally intelligent LLM-Augmented pedagogical AI conversational agent," *Educ Inf Technol*, Sept. 2025, doi: 10.1007/s10639-025-13755-7.
- [5] J. Kim, "Types of teacher-AI collaboration in K-12 classroom instruction: Chinese teachers' perspective," *Educ Inf Technol*, vol. 29, no. 13, pp. 17433–17465, Sept. 2024, doi: 10.1007/s10639-024-12523-3

Appendix: Source code

app.py

```
import os
import json
import datetime
from flask import Flask, render_template, request, Response, jsonify,
send_file, stream_with_context
from dotenv import load_dotenv
from utils.ai import stream_with_accumulation, build_lesson_plan_prompt,
build_progress_report_prompt
from utils.pdf_gen import create_pdf_report, generate_progress_pdf
from utils.emailer import send_email
from models import SessionLocal, Student, LessonPlan, ProgressReport
import pandas as pd
from apscheduler.schedulers.background import BackgroundScheduler

load_dotenv()
REPORTS_DIR = os.getenv("REPORTS_DIR", "reports")
os.makedirs(REPORTS_DIR, exist_ok=True)

app = Flask(__name__, template_folder="templates")
scheduler = BackgroundScheduler()
scheduler.start()

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/ping")
def ping():
    return jsonify({"message": "✅ Flask is alive"})

# ----- Stream Lesson Plan -----
@app.route("/stream-lesson-plan", methods=["POST"])
def stream_lesson_plan_api():
    data = request.get_json(force=True, silent=True) or {}
    subject = data.get("subject")
    topic = data.get("topic")
    student_name = data.get("studentName", "Student")
    if not subject or not topic:
        return jsonify({"error": "subject and topic required"}), 400

    prompt = build_lesson_plan_prompt(
        subject=subject,
        topic=topic,
        grade=data.get("grade"),
        date_time=data.get("dateTime"),
        duration=data.get("duration"),
        teacher_notes=data.get("teacherNotes")
    )

    def generate():
        accumulated = ""
        for chunk in stream_with_accumulation(prompt):
```

```

        if isinstance(chunk, tuple):
            _, accumulated = chunk
        else:
            yield chunk

    filename =
f"{student_name}_{subject}_lesson_{int(datetime.datetime.utcnow().timestamp())}
.pdf"
    path = os.path.join(REPORTS_DIR, filename)
    create_pdf_report(f"Lesson Plan: {subject} - {topic}", accumulated,
path)

    db = SessionLocal()
    try:
        student = db.query(Student).filter_by(name=student_name).first()
        if not student:
            student = Student(name=student_name, grade=data.get("grade",
"N/A"))
            db.add(student); db.commit(); db.refresh(student)
        lesson = LessonPlan(subject=subject, content=accumulated,
pdf_filename=filename, student=student)
        db.add(lesson); db.commit()
    finally:
        db.close()

    yield "\n" + json.dumps({
        "done": True,
        "pdf_path": f"/download/{filename}",
        "studentName": student_name
    })

    return Response(stream_with_context(generate()), mimetype="text/plain")

# ----- Stream Progress Report -----
@app.route("/stream-progress-report", methods=["POST"])
def stream_progress_report_api():
    data = request.get_json(force=True, silent=True) or {}
    student_name = data.get("studentName")
    perf = data.get("performanceData", [])
    if not student_name or not perf:
        return jsonify({"error": "Missing studentName or performanceData"}), 400

    print(f"[DEBUG] Progress report request for {student_name}: {perf}")

    prompt = build_progress_report_prompt(
        student_name, perf,
        weaknesses=data.get("improvementAreas", ""),
        next_steps=data.get("nextSteps", "")
    )

    def generate():
        accumulated = ""
        for chunk in stream_with_accumulation(prompt):
            if isinstance(chunk, tuple):
                _, accumulated = chunk
            else:
                yield chunk

```

```

        # Parse scores robustly
        scores = []
        for r in perf:
            s_raw = str(r.get("score", ""))
            digits = "".join(ch for ch in s_raw if ch.isdigit())
            if digits:
                try:
                    scores.append(int(digits))
                except ValueError:
                    continue

        filename =
f"{student_name}_progress_{int(datetime.datetime.utcnow().timestamp())}.pdf"
        path = os.path.join(REPORTS_DIR, filename)
        generate_progress_pdf(student_name, "AI Analysis",
data.get("improvementAreas", ""), scores, path)

        db = SessionLocal()
        try:
            student = db.query(Student).filter_by(name=student_name).first()
            if not student:
                student = Student(name=student_name, grade=data.get("grade",
"N/A"))
                db.add(student); db.commit(); db.refresh(student)
            report = ProgressReport(summary=accumulated, pdf_filename=filename,
student=student)
            db.add(report); db.commit()
        finally:
            db.close()

        yield "\n" + json.dumps({
            "done": True,
            "pdf_path": f"/download/{filename}",
            "studentName": student_name
        })

    return Response(stream_with_context(generate()), mimetype="text/plain")

# ----- Download File -----
@app.route("/download/<path:filename>")
def download_file(filename):
    path = os.path.join(REPORTS_DIR, filename)
    if os.path.exists(path):
        return send_file(path, as_attachment=True)
    return jsonify({"error": "File not found"}), 404

# ----- Send Report to Parent -----
@app.route("/send-report-to-parent", methods=["POST"])
def send_report_to_parent():
    data = request.json or {}
    required = ["studentName", "parentEmail", "reportText"]
    if not all(k in data for k in required):
        return jsonify({"status": "error", "message": "Missing fields"}), 400
    try:
        attachment = None
        pdf_path = data.get("pdfPath")

```



```

        if pdf_path and pdf_path.startswith("/download/"):
            attachment = os.path.join(REPORTS_DIR, os.path.basename(pdf_path))
            if not os.path.exists(attachment): attachment = None
            send_email(data["parentEmail"], f"Progress Report for {data['studentName']}", data["reportText"], attachment)
            return jsonify({"status": "success", "message": "Sent"})
        except Exception as e:
            return jsonify({"status": "error", "message": str(e)}), 500

# ----- Schedule Email -----
@app.route("/schedule-email", methods=["POST"])
def schedule_email():
    data = request.json or {}
    run_at = data.get("run_at")
    to = data.get("to")
    subject = data.get("subject", "Scheduled Report")
    body = data.get("body", "")
    attachment = data.get("attachment")

    if not run_at or not to:
        return jsonify({"status": "error", "message": "run_at and to required"}),
400
    try:
        run_dt = datetime.datetime.fromisoformat(run_at)
    except Exception as e:
        return jsonify({"status": "error", "message": "invalid datetime format"}),
400

    def job_func(to, subject, body, attachment):
        att_path = None
        if attachment and attachment.startswith("/download/"):
            att_path = os.path.join(REPORTS_DIR, os.path.basename(attachment))
            if not os.path.exists(att_path): att_path = None
        try:
            send_email(to, subject, body, att_path)
            print(f"Scheduled email sent to {to} at {datetime.datetime.utcnow()}")
        except Exception as e:
            print("Scheduled email failed:", e)

    job = scheduler.add_job(job_func, trigger='date', run_date=run_dt, args=[to, subject, body, attachment])
    return jsonify({"status": "scheduled", "job_id": job.id})

# ----- Upload Excel & Dashboard -----
@app.route("/upload-excel", methods=["POST"])
def upload_excel():
    if 'file' not in request.files:
        return jsonify({"error": "No file uploaded"}), 400
    f = request.files['file']
    try:
        df = pd.read_excel(f)
    except Exception as e:
        return jsonify({"error": "Could not read Excel: " + str(e)}), 400

    cols = {c.lower().strip(): c for c in df.columns}
    def find_col(possible):

```

```

        for p in possible:
            if p in cols: return cols[p]
        return None

    student_col = find_col(['student', 'name'])
    subject_col = find_col(['subject'])
    score_col = find_col(['score', 'marks'])
    date_col = find_col(['date', 'assessment date', 'exam date'])
    topic_col = find_col(['topic', 'chapter'])

    if not student_col or not score_col:
        return jsonify({"error": "Excel must include Student (or Name) and Score columns"}), 400

    # Clean scores
    def to_num(v):
        try:
            s = str(v)
            digits = "".join(ch for ch in s if (ch.isdigit() or ch=='.'))
            return float(digits) if digits else None
        except:
            return None

    df['__score'] = df[score_col].apply(to_num)
    df['__subject'] = df[subject_col].fillna("General") if subject_col else
pd.Series(["General"]*len(df))

    # Subject averages
    subject_averages =
df.groupby('__subject')['__score'].mean().round(1).fillna(0).to_dict()
    overall_avg = float(df['__score'].mean().round(1))

    # Distribution
    bins = {'A (90+)':0, 'B (75-89)':0, 'C (60-74)':0, 'D (<60)':0}
    for v in df['__score'].dropna():
        if v>=90: bins['A (90+)']+=1
        elif v>=75: bins['B (75-89)']+=1
        elif v>=60: bins['C (60-74)']+=1
        else: bins['D (<60)']+=1

    # Per-student scores
    if date_col:
        try:
            df['__date'] = pd.to_datetime(df[date_col], errors='coerce')
            df = df.sort_values('__date')
        except:
            pass

    students = {}
    for _, row in df.iterrows():
        name = str(row[student_col])
        students.setdefault(name, []).append({'score': (row['__score'] or 0),
'subject': row['__subject'], 'topic': row.get(topic_col, "")})

    # Top weak topics
    topic_scores = {}
    for _, row in df.iterrows():

```

```

        topic = row.get(topic_col, "General") if topic_col else "General"
        score = row['__score']
        if score is not None:
            topic_scores.setdefault(topic, []).append(score)
        topic_averages = {t: sum(v)/len(v) for t,v in topic_scores.items() if v}
        top_weak_topics = sorted(topic_averages.items(), key=lambda x: x[1])[:5]

    resp = {
        "overall_average": overall_avg,
        "subject_averages": {k: float(v) for k,v in subject_averages.items()},
        "distribution": bins,
        "students": students,
        "top_weak_topics": [{"topic": t, "avg_score": round(s,1)} for t,s in
top_weak_topics]
    }
    return jsonify(resp)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.getenv("PORT", 5000)), debug=True)

```

ai.py

```

import os, datetime
import ollama
MODEL = os.environ.get("OLLAMA_MODEL", "llama3")

def stream_with_accumulation(prompt: str):
    full_text = ""
    stream = ollama.chat(model=MODEL,
messages=[{"role":"user","content":prompt}], stream=True)
    for chunk in stream:
        content = chunk.get("message", {}).get("content")
        if content:
            full_text += content
            yield content
        if chunk.get("done"):
            break
    yield (None, full_text)

def build_lesson_plan_prompt(subject, topic, grade=None, date_time=None,
duration=None, teacher_notes=None):
    return f"""
You are an AI teaching assistant. Create a clear, printable lesson plan.

Subject: {subject}
Topic: {topic}
Grade: {grade or 'Not specified'}
Date/Time: {date_time or 'Not specified'}
Duration: {duration or 'Not specified'}
Teacher Notes / Preferences: {teacher_notes or 'None'}

Please produce:
1) Header: date, time, lesson title, grade
2) Lesson Procedures (The "How-To") - step by step
3) Assessment ideas (formative / summative)
4) Homework / Independent Practice
5) Differentiation tips (if applicable)

Keep it teacher-friendly and succinct.
"""

def build_progress_report_prompt(student_name, performance_data, weaknesses='',
next_steps=''):
    perf_lines = "\n".join([f"- {p.get('topic')}: {p.get('score')}" for p in
performance_data])
    return f"""
You are an assistant writing a progress report for a student.

Student: {student_name}
Performance:
{perf_lines}

Weaknesses: {weaknesses or 'N/A'}
Next steps: {next_steps or 'N/A'}

Produce:
1) Short summary (1-2 paragraphs)
2) Subject-specific performance bullets

```

```
3) Recommendations & measurable goals
4) "Report generated on" timestamp
"""
```

emailer.py

```
import os, smtplib
```

```

from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
from dotenv import load_dotenv

load_dotenv()
EMAIL_HOST = os.getenv("EMAIL_HOST", "smtp.gmail.com")
EMAIL_PORT = int(os.getenv("EMAIL_PORT", 587))
EMAIL_ADDRESS = os.getenv("EMAIL_ADDRESS")
EMAIL_PASSWORD = os.getenv("EMAIL_PASSWORD")

def send_email(to, subject, body, attachment=None):
    if not EMAIL_ADDRESS or not EMAIL_PASSWORD:
        raise RuntimeError("EMAIL_ADDRESS and EMAIL_PASSWORD must be set in
.env")

    msg = MIMEMultipart()
    msg["From"] = EMAIL_ADDRESS
    msg["To"] = to
    msg["Subject"] = subject
    msg.attach(MIMEText(body, "plain"))

    if attachment and os.path.exists(attachment):
        part = MIMEBase("application", "octet-stream")
        with open(attachment, "rb") as f:
            part.set_payload(f.read())
        encoders.encode_base64(part)
        part.add_header("Content-Disposition", f'attachment;
filename="{os.path.basename(attachment)}"')
        msg.attach(part)

    with smtplib.SMTP(EMAIL_HOST, EMAIL_PORT) as server:
        server.starttls()
        server.login(EMAIL_ADDRESS, EMAIL_PASSWORD)
        server.send_message(msg)

```

pdf_gen.py

```
import os
```

```

from fpdf import FPDF
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

FONT_PATH = os.path.join("fonts", "DejaVuSans.ttf")

def _setup_font(pdf, bold=False, size=12):
    if os.path.exists(FONT_PATH):
        pdf.add_font("DejaVu", 'B' if bold else '', FONT_PATH, uni=True)
        pdf.set_font("DejaVu", 'B' if bold else '', size)
    else:
        pdf.set_font("Arial", 'B' if bold else '', size)

def create_pdf_report(title, content, filename="report.pdf"):
    pdf = FPDF()
    pdf.add_page()
    _setup_font(pdf, bold=True, size=14)
    pdf.multi_cell(0, 10, title, align="C")
    pdf.ln(5)
    _setup_font(pdf, size=12)
    pdf.multi_cell(0, 8, content.replace("-", "-"))
    pdf.output(filename)
    return filename

def generate_progress_pdf(student_name, strengths, weaknesses, scores,
filename="progress.pdf"):
    chart_file = filename + ".png"
    plt.figure()
    if scores:
        plt.plot(scores, marker="o")
    plt.title(f"{student_name} Progress")
    plt.xlabel("Assessment")
    plt.ylabel("Score")
    plt.tight_layout()
    plt.savefig(chart_file)
    plt.close()

    pdf = FPDF()
    pdf.add_page()
    _setup_font(pdf, bold=True, size=14)
    pdf.cell(0, 10, f"Progress Report - {student_name}", ln=True, align="C")
    pdf.ln(5)
    _setup_font(pdf, size=12)
    pdf.multi_cell(0, 8, f"Strengths: {strengths}\n\nWeaknesses: {weaknesses}")
    if os.path.exists(chart_file):
        pdf.image(chart_file, x=25, w=160)
        os.remove(chart_file)
    pdf.output(filename)
    return filename

```

index.html

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width,initial-scale=1" />
<title>Mentorly AI: Your Co-pilot in the Classroom</title>
<script src="https://cdn.tailwindcss.com"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body class="bg-gray-100 min-h-screen p-6 flex items-start justify-center">
<div class="container max-w-5xl bg-white rounded-xl p-6 shadow-xl">

  <header class="text-center mb-6">
    <h1 class="text-3xl font-bold text-blue-700">Mentorly AI: Your Co-pilot in
the Classroom</h1>
    <p class="text-gray-600 mt-1">Focus on teaching – we'll handle the rest</p>
  </header>

  <!-- Navigation -->
  <div class="grid grid-cols-3 gap-4 mb-6">
    <button id="navLesson" class="p-4 bg-blue-50 rounded shadow">
      <h3 class="font-semibold">Lesson Planning</h3>
      <p class="text-sm text-gray-600">Craft custom lessons & resources in
minutes.</p>
    </button>
    <button id="navReports" class="p-4 bg-green-50 rounded shadow">
      <h3 class="font-semibold">Smart Reports</h3>
      <p class="text-sm text-gray-600">Automate and track progress</p>
    </button>
    <button id="navDashboard" class="p-4 bg-purple-50 rounded shadow">
      <h3 class="font-semibold">Personalised student support</h3>
      <p class="text-sm text-gray-600">Upload an Excel to visualize class
performance</p>
    </button>
  </div>

  <!-- LESSON PLANNING -->
  <section id="lessonSection" class="mb-6 hidden">
    <h2 class="text-xl font-bold text-blue-700 mb-3">Lesson Planner</h2>
    <div class="grid grid-cols-2 gap-3 mb-3">
      <input id="studentName" placeholder="Student Name (optional)" class="p-2
border rounded" />
      <input id="subject" placeholder="Subject (e.g., Math)" class="p-2 border
rounded" />
      <input id="grade" placeholder="Grade (e.g., 8)" class="p-2 border rounded"
/>
      <input id="topic" placeholder="Topic (e.g., Fractions)" class="p-2 border
rounded" />
      <input id="lessonDateTime" type="datetime-local" class="p-2 border
rounded" />
      <input id="duration" placeholder="Duration (e.g., 40 mins)" class="p-2
border rounded" />
    </div>
    <textarea id="teacherNotes" class="w-full border p-2 rounded mb-3"
placeholder="Optional: Add teacher notes"></textarea>
    <div class="flex gap-3">
      <button onclick="generateLessonPlan()" id="lessonBtn" class="bg-blue-600
text-white px-4 py-2 rounded">Generate Lesson Plan</button>

```



```

        <a id="downloadLessonLink" class="hidden ml-2 px-4 py-2 rounded
bg-gray-200" href="#" download>⬇ Download Lesson PDF</a>
    </div>
    <div id="lessonLoading" class="hidden mt-3">⌚ Generating lesson...</div>
    <div id="lessonOutput" class="hidden bg-blue-50 mt-3 p-3 rounded
whitespace-pre-wrap"></div>
</section>

<!-- PROGRESS REPORTS -->
<section id="reportSection" class="mb-6 hidden">
    <h2 class="text-xl font-bold text-green-700 mb-3">Progress Report</h2>
    <div class="grid grid-cols-2 gap-3 mb-3">
        <input id="reportStudent" placeholder="Student Name" class="p-2 border
rounded" />
        <input id="reportSubject" placeholder="Subject" class="p-2 border rounded"
/>
    </div>
    <div id="perfRows" class="mb-2">
        <div class="flex gap-2 mb-1">
            <input class="p-2 border rounded flex-1 perf-topic" placeholder="Topic"
/>
            <input class="p-2 border rounded perf-score" placeholder="Score (e.g.,
78%)" />
        </div>
    </div>
    <button onclick="addPerfRow()" class="mb-3 text-sm text-gray-600">+ Add
topic row</button>
    <textarea id="reportNotes" class="w-full border p-2 rounded mb-3"
placeholder="Improvement areas / next steps (optional)"></textarea>
    <div class="flex gap-3">
        <button onclick="generateProgressReport()" id="reportBtn"
class="bg-green-600 text-white px-4 py-2 rounded">Generate Report</button>
        <a id="downloadReportLink" class="hidden ml-2 px-4 py-2 rounded
bg-gray-200" href="#" download>⬇ Download Report PDF</a>
    </div>
    <div class="mt-3">
        <label class="inline-block mr-2">Send now or schedule?</label>
        <input id="parentEmail" type="email" placeholder="Parent email" class="p-2
border rounded mr-2" />
        <button onclick="sendReportNow()" class="bg-purple-600 text-white px-3
py-2 rounded">Send Now</button>
        <input id="scheduleDatetime" type="datetime-local" class="p-2 border
rounded ml-2" />
        <button onclick="scheduleReport()" class="bg-yellow-500 text-white px-3
py-2 rounded ml-1">Schedule</button>
    </div>
    <div id="sendStatus" class="mt-2 text-sm"></div>
    </div>
    <div id="reportLoading" class="hidden mt-3">⌚ Generating report...</div>
    <div id="reportOutput" class="hidden bg-green-50 mt-3 p-3 rounded
whitespace-pre-wrap"></div>
</section>

<!-- DASHBOARD: Excel upload & charts -->
<section id="dashboardSection" class="hidden">
    <h2 class="text-xl font-bold text-purple-700 mb-3">Student Dashboard</h2>
    <p class="text-sm text-gray-600 mb-2">Upload an Excel (columns: Student,
Subject, Topic, Score, Date [optional])</p>

```

```

<input id="excelFile" type="file" accept=".xls,.xlsx" class="mb-3" />
<button onclick="uploadExcel()" class="bg-purple-600 text-white px-4 py-2
rounded">Upload & Analyze</button>
<div id="dashboardCharts" class="mt-4 hidden">
  <canvas id="classAvgChart" class="mb-4"></canvas>
  <canvas id="distChart" class="mb-4" style="max-width:500px"></canvas>
  <div id="studentCharts" class="grid grid-cols-2 gap-4"></div>
</div>
<div id="dashboardStatus" class="mt-2 text-sm"></div>
</section>
</div>

<script>
// Navigation
const g = id => document.getElementById(id);
g('navLesson').onclick = () => showSection('lessonSection');
g('navReports').onclick = () => showSection('reportSection');
g('navDashboard').onclick = () => showSection('dashboardSection');
function showSection(id){
  ['lessonSection','reportSection','dashboardSection'].forEach(s =>
g(s).classList.add('hidden'));
  g(id).classList.remove('hidden'); window.scrollTo({top:0,
behavior:'smooth'});
}

// Lesson planner & progress report streaming helpers
let lastReport=null; let charts=[];

async function streamAPI(endpoint, payload, opts){
  const loading=g(opts.loadingId), output=g(opts.outputId),
    downloadLink=g(opts.downloadLinkId), btn=g(opts.buttonId);
  loading.classList.remove('hidden'); btn.disabled=true;
output.classList.remove('hidden'); output.innerHTML="";
  downloadLink.classList.add('hidden');

  try{
    const res = await fetch(endpoint,{
      method:'POST', headers:{'Content-Type':'application/json'},
body:JSON.stringify(payload)
    });
    if(!res.ok || !res.body) throw new Error("HTTP "+res.status);
    const reader=res.body.getReader(), decoder=new TextDecoder();
    let done=false, accumulated="";
    while(!done){
      const {value, done:readerDone}=await reader.read();
      if(readerDone){ done=true; break; }
      accumulated+=decoder.decode(value,{stream:true});
      output.innerHTML=accumulated.replace(/\n/g,"<br>");
    }
    const
jsonMatch=accumulated.match(/\{([\s\S]*"done"\s*:\s*true[\s\S]*\)\s*$/);
    if(jsonMatch){
      const meta=JSON.parse(jsonMatch[0]);
      const visible=accumulated.replace(jsonMatch[0],"");
      output.innerHTML=visible.replace(/\n/g,"<br>");
    }
  }
}

```

```

        if(meta.pdf_path){ downloadLink.href=meta.pdf_path;
downloadLink.classList.remove('hidden'); lastReport=lastReport||{};
lastReport.pdfPath=meta.pdf_path; }
        if(meta.studentName)
lastReport=Object.assign(lastReport||{},{studentName:meta.studentName});
        if(visible)
lastReport=Object.assign(lastReport||{},{reportText:visible});
        } else lastReport=lastReport||{}; lastReport.reportText=accumulated;
    }catch(e){ console.error("Stream failed",e); alert("✖ Stream failed:
"+e.message); }
    finally{ loading.classList.add('hidden'); btn.disabled=false; }
}

function addPerfRow(){
    const row=document.createElement('div'); row.className="flex gap-2 mb-1";
    row.innerHTML='<input class="p-2 border rounded flex-1 perf-topic"
placeholder="Topic"/><input class="p-2 border rounded perf-score"
placeholder="Score"/>';
    g('perfRows').appendChild(row);
}

function generateLessonPlan(){
    const payload={
        studentName:g('studentName').value, subject:g('subject').value,
        grade:g('grade').value, topic:g('topic').value,
        lessonDateTime:g('lessonDateTime').value, duration:g('duration').value,
        teacherNotes:g('teacherNotes').value
    };
    streamAPI('/stream-lesson', payload,{loadingId:'lessonLoading',
outputId:'lessonOutput', downloadLinkId:'downloadLessonLink',
buttonId:'lessonBtn'});
}

function generateProgressReport(){
    const
topics=[...document.getElementsByClassName('perf-topic')].map((e,i)=>({topic:e.
value,score:document.getElementsByClassName('perf-score')[i].value}));
    const
payload={studentName:g('reportStudent').value,subject:g('reportSubject').value,
topics,notes:g('reportNotes').value};
    streamAPI('/stream-progress-report', payload,{loadingId:'reportLoading',
outputId:'reportOutput', downloadLinkId:'downloadReportLink',
buttonId:'reportBtn'});
}

function sendReportNow(){ g('sendStatus').innerText="✉ Report sent to
"+g('parentEmail').value; }
function scheduleReport(){ g('sendStatus').innerText="🕒 Report scheduled for
"+g('scheduleDatetime').value; }

async function uploadExcel(){
    const file=g('excelFile').files[0]; if(!file){ alert("Select Excel!");
return; }
    g('dashboardStatus').innerText="🕒 Uploading and analyzing...";
    const fd=new FormData(); fd.append("file",file);
    try{
        const res=await fetch('/upload-excel',{method:'POST',body:fd});

```

```

const data=await res.json();
g('dashboardStatus').innerText="✅ Analysis complete";
g('dashboardCharts').classList.remove('hidden');
// Clear previous charts
charts.forEach(c=>c.destroy()); charts=[];
const ctx1=g('classAvgChart').getContext('2d');
charts.push(new
Chart(ctx1,{type:'bar',data:{labels:Object.keys(data.subject_averages),datasets
:[{label:'Avg
Score',data:Object.values(data.subject_averages),backgroundColor:'#8b5cf6'}]},o
ptions:{responsive:true}}));
const ctx2=g('distChart').getContext('2d');
charts.push(new
Chart(ctx2,{type:'pie',data:{labels:Object.keys(data.distribution),datasets:[{d
ata:Object.values(data.distribution),backgroundColor:['#f87171','#fbbf24','#34d
399','#60a5fa','#a78bfa']}]},options:{responsive:true}}));
} catch(e){ console.error(e); g('dashboardStatus').innerText="❌ Upload
failed: "+e.message; }
}
</script>
</body>
</html>

```